

EXPERT SYSTEMS

COMPLETE PRACTICAL

GUIDE

*From Knowledge Representation to
Intelligent Systems*

Build AI Systems That Think Like Human Experts

Comprehensive Course Coverage:

Introduction to Expert Systems and AI

Knowledge Representation Techniques

Rule-Based Systems Architecture

Inference Engines (Forward & Backward Chaining)

Uncertainty Handling and Fuzzy Logic

Knowledge Acquisition Methods

CLIPS Programming Language

Python Expert System Development

Medical Diagnosis Systems

Financial Advisory Systems

Fault Diagnosis and Troubleshooting

Natural Language Processing Integration

Expert System Development Lifecycle

Real-World Case Studies

Hands-On Implementation Projects

TABLE OF CONTENTS

PART 1: EXPERT SYSTEMS FUNDAMENTALS

- Chapter 1: Introduction to Expert Systems
- Chapter 2: History and Evolution
- Chapter 3: Expert Systems vs Traditional Programs
- Chapter 4: Components of Expert Systems

PART 2: KNOWLEDGE REPRESENTATION

- Chapter 5: Knowledge Representation Schemes
- Chapter 6: Production Rules (IF-THEN)
- Chapter 7: Semantic Networks
- Chapter 8: Frames and Objects
- Chapter 9: Logic and Predicates

PART 3: INFERENCE ENGINES

- Chapter 10: Inference Mechanisms
- Chapter 11: Forward Chaining
- Chapter 12: Backward Chaining
- Chapter 13: Conflict Resolution Strategies

PART 4: UNCERTAINTY AND REASONING

- Chapter 14: Handling Uncertainty
- Chapter 15: Certainty Factors
- Chapter 16: Bayesian Networks
- Chapter 17: Fuzzy Logic Systems

Chapter 18: Dempster-Shafer Theory

PART 5: KNOWLEDGE ACQUISITION

Chapter 19: Knowledge Engineering Process

Chapter 20: Knowledge Elicitation Techniques

Chapter 21: Knowledge Validation

Chapter 22: Knowledge Base Maintenance

PART 6: EXPERT SYSTEM DEVELOPMENT

Chapter 23: Development Lifecycle

Chapter 24: Requirements Analysis

Chapter 25: Knowledge Base Design

Chapter 26: System Architecture

Chapter 27: User Interface Design

PART 7: CLIPS PROGRAMMING

Chapter 28: Introduction to CLIPS

Chapter 29: Facts and Rules in CLIPS

Chapter 30: Pattern Matching

Chapter 31: Functions and Modules

Chapter 32: Complete CLIPS Applications

PART 8: PYTHON EXPERT SYSTEMS

Chapter 33: Python for Expert Systems

Chapter 34: Experta Library

Chapter 35: PyKnow Framework

Chapter 36: Building Custom Inference Engines

PART 9: REAL-WORLD APPLICATIONS

Chapter 37: Medical Diagnosis Systems

Chapter 38: Financial Planning Systems

Chapter 39: Equipment Fault Diagnosis

Chapter 40: Legal Advisory Systems

Chapter 41: Agricultural Expert Systems

PART 10: PRACTICAL PROJECTS

Project 1: Medical Diagnosis Expert System

Project 2: Car Troubleshooting System

Project 3: Investment Advisor

Project 4: Computer Configuration Assistant

Project 5: Crop Disease Identification System

PART 11: ADVANCED TOPICS

Chapter 42: Neural-Symbolic Integration

Chapter 43: Explanation Facilities

Chapter 44: Learning Expert Systems

Chapter 45: Distributed Expert Systems

APPENDICES

Appendix A: CLIPS Reference Guide

Appendix B: Python Libraries for ES

Appendix C: Famous Expert Systems

Appendix D: Resources and Tools

CHAPTER 1: INTRODUCTION TO EXPERT SYSTEMS

1.1 What is an Expert System?

An Expert System (ES) is a computer program that emulates the decision-making ability of a human expert in a specific domain.

Simple Definition:

A computer system that solves complex problems by reasoning through knowledge, like a human expert would.

Real-World Analogy:

Think of visiting a doctor:

- You describe symptoms
- Doctor asks specific questions
- Doctor applies medical knowledge
- Doctor makes diagnosis
- Doctor recommends treatment

An expert system does this systematically using:

- Knowledge from human experts
- Reasoning mechanisms
- User interaction

Key Characteristics:

1. High Performance

- Solves problems at expert level
- Handles complex scenarios
- Provides accurate solutions

2. Domain-Specific

- Specialized in one area (medicine, finance, etc.)
- Deep knowledge of that domain
- Not general-purpose AI

3. Explains Reasoning

- Can explain how it reached conclusion
- Shows reasoning path
- Justifies recommendations

4. Uses Symbolic Reasoning

- Works with rules and logic
- Not statistical/numerical only
- Mimics human thought process

1.2 Famous Expert Systems

Historical Expert Systems:

1. MYCIN (1970s)

- Domain: Medical diagnosis
- Purpose: Diagnose blood infections
- Achievement: Matched human expert performance
- Used certainty factors for uncertainty
- 600 rules, 85% accuracy

2. DENDRAL (1965)

- Domain: Chemistry
- Purpose: Identify molecular structures
- First successful expert system
- Used mass spectrometry data

3. XCON (R1) (1980s)

- Domain: Computer configuration
- Purpose: Configure VAX computers
- Deployed at Digital Equipment Corporation
- Saved \$40 million annually
- 10,000+ rules

4. PROSPECTOR (1970s)

- Domain: Mineral exploration
- Purpose: Locate ore deposits
- Discovered molybdenum deposit worth \$100 million

5. CLIPS (NASA, 1980s)

- General-purpose expert system shell
- Still widely used today
- Public domain, free to use

1.3 Applications of Expert Systems

Expert systems are used in diverse fields:

1. Medicine and Healthcare

- Disease diagnosis
- Treatment planning
- Drug interaction checking
- Medical image analysis

Example: DXplain (diagnostic system with 2,000+ diseases)

2. Finance and Business

- Investment advice
- Credit approval
- Fraud detection
- Tax planning

Example: Credit scoring systems

3. Manufacturing

- Equipment fault diagnosis
- Process control
- Quality assurance
- Production scheduling

4. Engineering

- Design automation
- Structural analysis
- Circuit diagnosis

5. Agriculture

- Crop disease identification
- Pest management
- Soil analysis
- Irrigation planning

6. Law and Legal Services

- Legal research
- Case analysis
- Contract review

7. Customer Service

- Technical support
- Help desk systems
- Product recommendations
- Troubleshooting guides

1.4 Advantages and Limitations

ADVANTAGES:

- ✓ Preserves Expert Knowledge
 - Captures expertise before retirement
 - Knowledge doesn't leave with expert
- ✓ Consistent Decisions
 - Same input = same output
 - No fatigue or emotions
- ✓ Available 24/7
 - Always accessible
 - No vacation or sick days
- ✓ Cost-Effective
 - One system serves many users
 - Reduces need for multiple experts
- ✓ Training Tool
 - Teaches novices
 - Explains reasoning

LIMITATIONS:

- X Domain-Limited
 - Works only in specific area
 - No common sense
- X Knowledge Acquisition Bottleneck
 - Difficult to extract expert knowledge
 - Time-consuming process

X Maintenance Challenges

- Knowledge needs updating
- Rules can become outdated

X Cannot Learn from Experience

- Traditional ES don't improve
- Need manual updates

X Lacks Creativity

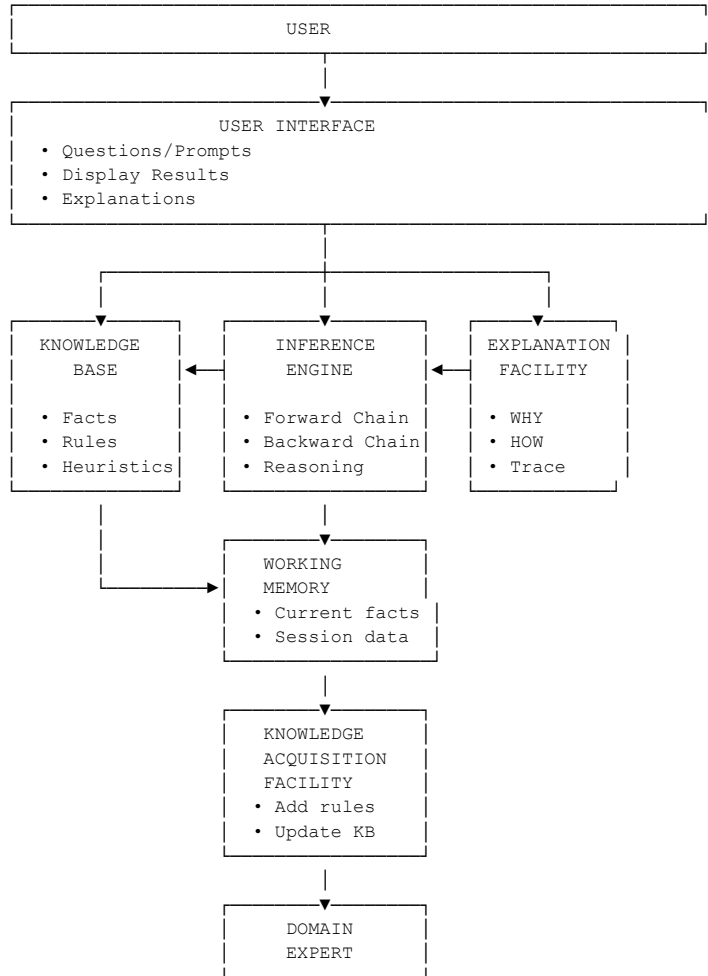
- Can't think outside rules
- No intuition or hunches

CHAPTER 4: COMPONENTS OF EXPERT SYSTEMS

4.1 Architecture Overview

An expert system consists of five main components:

EXPERT SYSTEM ARCHITECTURE:



COMPONENT DETAILS:

1. KNOWLEDGE BASE

Contains domain knowledge in the form of:

Facts:

- Definite statements about the domain
- Example: "The patient has a fever"
- Example: "The car won't start"

Rules (Production Rules):

- IF-THEN statements
- Format: IF <condition> THEN <conclusion>

Example Rule (Medical):

```
IF patient has fever
  AND patient has cough
  AND patient has body-ache
THEN patient likely has flu
```

Example Rule (Automotive):

```
IF engine-cranks = no
  AND battery-voltage < 12
THEN problem = dead-battery
```

Heuristics:

- Rules of thumb from experts
- "Best practices"
- Example: "If two symptoms match, check related symptoms first"

2. INFERENCE ENGINE

The "brain" of the expert system.
Performs logical reasoning using knowledge base.

Two Main Strategies:

A) Forward Chaining (Data-Driven)

- Start with known facts
- Apply rules to derive new facts
- Continue until goal reached
- Used for: Planning, monitoring, control

Example Flow:

1. Known: Car won't start
2. Rule: IF won't-start THEN check-battery
3. New fact: Check battery
4. Known: Battery voltage = 11V
5. Rule: IF voltage < 12V THEN battery-dead
6. Conclusion: Replace battery

B) Backward Chaining (Goal-Driven)

- Start with hypothesis/goal
- Try to prove it using rules
- Work backwards to find supporting facts
- Used for: Diagnosis, classification

Example Flow:

1. Goal: Does patient have flu?
2. Rule: IF fever AND cough AND body-ache THEN flu
3. Check: Does patient have fever? (Ask user)
4. Check: Does patient have cough? (Ask user)
5. Check: Does patient have body-ache? (Ask user)
6. If all YES → Conclude: Patient has flu

Conflict Resolution:

When multiple rules can fire:

- Specificity: More specific rules first
- Recency: Recently added facts first
- Priority: User-assigned priorities

3. WORKING MEMORY

Temporary storage for current session.

Contains:

- Initial facts from user
- Derived facts from reasoning
- Current state of consultation
- Intermediate results

Example Working Memory During Session:

Initial State:

- patient-age = 45
- patient-symptom = fever
- patient-symptom = cough

After Inference:

- patient-age = 45
- patient-symptom = fever
- patient-symptom = cough
- check-flu = yes (derived)
- possible-diagnosis = flu (derived)
- recommend-test = rapid-flu-test (derived)

4. USER INTERFACE

Interaction between user and system.

Functions:

- Ask questions
- Display results
- Show explanations
- Accept input
- Format output

Example Interaction:

System: What symptoms does the patient have?

User: Fever and cough

System: How high is the fever?

User: 102°F

System: How long has the patient had these symptoms?

User: 3 days

System: [Reasoning...]

System: DIAGNOSIS: Likely influenza
CONFIDENCE: 85%

RECOMMENDED ACTIONS:

1. Rest and fluids
2. Take fever reducer
3. Consider antiviral medication
4. Monitor for 48 hours
5. Seek medical attention if symptoms worsen

5. EXPLANATION FACILITY

Explains system's reasoning to user.

Two Main Questions:

WHY?

"Why is the system asking this question?"

Example:

User: WHY are you asking about fever duration?

System: I am trying to determine if the patient has flu.

```
Rule 12 states:
  IF fever duration > 2 days
    AND fever > 101°F
    AND patient has cough
  THEN suspect influenza
```

HOW?

"How did the system reach this conclusion?"

Example:

User: HOW did you determine flu?

System: I concluded the patient has flu because:

1. Patient has fever (user input)
2. Fever is 102°F > 101°F (user input)
3. Fever duration is 3 days > 2 days (user input)
4. Patient has cough (user input)
5. Rule 12 fired: All conditions met
6. Confidence factor: 0.85 (based on symptom match)

6. KNOWLEDGE ACQUISITION FACILITY

Tools for building and maintaining knowledge base.

Functions:

- Add new rules
- Modify existing rules
- Delete outdated rules
- Validate consistency
- Test knowledge base

Example Rule Editor:

RULE EDITOR
Rule ID: R_FLU_001 Priority: 5
IF Conditions:
☒ fever = true
☒ cough = true
☒ body-ache = true
☐ runny-nose = true
THEN Conclusions:
• diagnosis = flu
• confidence = 0.85
• recommend = rest-fluids
[Save] [Test] [Cancel]

CHAPTER 11: FORWARD CHAINING

FORWARD CHAINING (DATA-DRIVEN REASONING)

Forward chaining starts with known facts and applies rules to derive new conclusions.

Flow: Facts → Rules → Conclusions

Algorithm:

1. Start with known facts in working memory
2. Find rules whose conditions match facts
3. Execute (fire) matching rules
4. Add conclusions to working memory
5. Repeat until goal reached or no rules match

EXAMPLE: ANIMAL IDENTIFICATION

Knowledge Base Rules:

Rule 1: IF animal has-hair THEN animal is-mammal
Rule 2: IF animal gives-milk THEN animal is-mammal
Rule 3: IF animal has-feathers THEN animal is-bird
Rule 4: IF animal flies AND animal lays-eggs THEN animal is-bird
Rule 5: IF animal is-mammal AND animal eats-meat THEN animal is-carnivore
Rule 6: IF animal is-mammal AND animal has-hooves THEN animal is-ungulate
Rule 7: IF animal is-mammal AND animal chews-cud THEN animal is-ungulate
Rule 8: IF animal is-carnivore AND animal has-tawny-color
AND animal has-dark-spots THEN animal is-cheetah
Rule 9: IF animal is-ungulate AND animal has-long-neck
AND animal has-long-legs THEN animal is-giraffe

PYTHON IMPLEMENTATION:

```
class ForwardChaining:
    def __init__(self):
        self.facts = set() # Working memory
        self.rules = []    # Knowledge base
        self.fired_rules = set() # Track which rules fired

    def add_fact(self, fact):
        """Add fact to working memory"""
        self.facts.add(fact)
        print(f"Added fact: {fact}")

    def add_rule(self, conditions, conclusion, name):
        """Add rule to knowledge base"""
        self.rules.append({
            'name': name,
            'conditions': conditions,
            'conclusion': conclusion
        })

    def check_conditions(self, conditions):
        """Check if all conditions are in facts"""
        return all(cond in self.facts for cond in conditions)

    def forward_chain(self):
        """Execute forward chaining"""
```

```

print("\n=== Starting Forward Chaining ===")

changes = True
iteration = 1

while changes:
    changes = False
    print(f"\n--- Iteration {iteration} ---")

    for rule in self.rules:
        # Skip already fired rules
        if rule['name'] in self.fired_rules:
            continue

        # Check if conditions are met
        if self.check_conditions(rule['conditions']):
            # Fire the rule
            if rule['conclusion'] not in self.facts:
                self.facts.add(rule['conclusion'])
                self.fired_rules.add(rule['name'])
                print(f"Fired: {rule['name']}")
                print(f"  Concluded: {rule['conclusion']}")
                changes = True

    iteration += 1

print("\n=== Forward Chaining Complete ===")

def get_conclusions(self):
    """Get final conclusions"""
    return self.facts

---

COMPLETE EXAMPLE:

# Create expert system
es = ForwardChaining()

# Add rules (knowledge base)
es.add_rule(
    ['has-hair'],
    'is-mammal',
    'Rule1: Hair → Mammal'
)

es.add_rule(
    ['gives-milk'],
    'is-mammal',
    'Rule2: Milk → Mammal'
)

es.add_rule(
    ['is-mammal', 'eats-meat'],
    'is-carnivore',
    'Rule5: Mammal + Meat → Carnivore'
)

es.add_rule(
    ['is-mammal', 'has-hooves'],
    'is-ungulate',
    'Rule6: Mammal + Hooves → Ungulate'
)

```

```

)

es.add_rule(
    ['is-carnivore', 'has-tawny-color', 'has-dark-spots'],
    'is-cheetah',
    'Rule8: Carnivore + Tawny + Spots → Cheetah'
)

es.add_rule(
    ['is-ungulate', 'has-long-neck', 'has-long-legs'],
    'is-giraffe',
    'Rule9: Ungulate + Long neck + Long legs → Giraffe'
)

# Add initial facts (observations)
print("=== Initial Facts (Observations) ===")
es.add_fact('has-hair')
es.add_fact('eats-meat')
es.add_fact('has-tawny-color')
es.add_fact('has-dark-spots')

# Execute forward chaining
es.forward_chain()

# Display results
print("\n=== Final Conclusions ===")
for fact in sorted(es.facts):
    print(f"    • {fact}")

---

OUTPUT:

=== Initial Facts (Observations) ===
Added fact: has-hair
Added fact: eats-meat
Added fact: has-tawny-color
Added fact: has-dark-spots

=== Starting Forward Chaining ===

--- Iteration 1 ---
Fired: Rule1: Hair → Mammal
Concluded: is-mammal

--- Iteration 2 ---
Fired: Rule5: Mammal + Meat → Carnivore
Concluded: is-carnivore

--- Iteration 3 ---
Fired: Rule8: Carnivore + Tawny + Spots → Cheetah
Concluded: is-cheetah

=== Forward Chaining Complete ===

=== Final Conclusions ===
    • eats-meat
    • has-dark-spots
    • has-hair
    • has-tawny-color
    • is-carnivore
    • is-cheetah

```

- is-mammal

WHEN TO USE FORWARD CHAINING:

- ✓ Planning and scheduling
- ✓ Monitoring and control systems
- ✓ Design and configuration
- ✓ Data analysis and interpretation
- ✓ When you have data and want conclusions

Examples:

- Process control: Given sensor readings, determine actions
- Network monitoring: Given events, identify problems
- Financial planning: Given income/expenses, create budget

CHAPTER 12: BACKWARD CHAINING

BACKWARD CHAINING (GOAL-DRIVEN REASONING)

Backward chaining starts with a goal (hypothesis) and works backwards to prove it.

Flow: Goal → Rules → Facts (questions)

Algorithm:

1. Start with goal to prove
2. Find rules that conclude the goal
3. Check if conditions are known
4. If not known, make conditions new sub-goals
5. Recursively prove sub-goals
6. Ask user for unknown facts

PYTHON IMPLEMENTATION:

```
class BackwardChaining:
    def __init__(self):
        self.facts = {}      # Known facts
        self.rules = []      # Knowledge base
        self.asked = set()   # Questions already asked

    def add_rule(self, conditions, conclusion, name):
        """Add rule to knowledge base"""
        self.rules.append({
            'name': name,
            'conditions': conditions,
            'conclusion': conclusion
        })

    def add_fact(self, fact, value):
        """Add known fact"""
        self.facts[fact] = value

    def ask_user(self, question):
        """Ask user for fact"""
        if question in self.asked:
            return self.facts.get(question, False)

        self.asked.add(question)

        # Format question nicely
        q = question.replace('-', ' ').replace('_', ' ')

        answer = input(f"Does the animal {q}? (yes/no): ").strip().lower()
        value = answer in ['yes', 'y', 'true']
        self.facts[question] = value

        return value

    def prove(self, goal, depth=0):
        """Try to prove goal using backward chaining"""
        indent = " " * depth
        print(f"{indent}Trying to prove: {goal}")

        # Check if already known
        if goal in self.facts:
            print(f"{indent}Already known: {goal} = {self.facts[goal]}")
            return self.facts[goal]

        # Find rules that conclude this goal
        applicable_rules = [r for r in self.rules if r['conclusion'] == goal]

        if not applicable_rules:
            # No rules found, ask user
            print(f"{indent}No rules found. Asking user...")
            return self.ask_user(goal)
```

```

        # Try each applicable rule
        for rule in applicable_rules:
            print(f"{indent}Trying rule: {rule['name']}")

            # Try to prove all conditions
            all_conditions_true = True

            for condition in rule['conditions']:
                # Recursively prove condition
                if not self.prove(condition, depth + 1):
                    all_conditions_true = False
                    break

            # If all conditions proved, conclude goal
            if all_conditions_true:
                print(f"{indent}✓ Proved: {goal}")
                self.facts[goal] = True
                return True

            # Couldn't prove with any rule
            print(f"{indent}X Cannot prove: {goal}")
            self.facts[goal] = False
            return False

---

COMPLETE EXAMPLE: MEDICAL DIAGNOSIS

# Create expert system
es = BackwardChaining()

# Add rules
es.add_rule(
    ['has-fever', 'has-cough', 'has-body-ache'],
    'has-flu',
    'Rule: Fever + Cough + Body-ache → Flu'
)

es.add_rule(
    ['has-fever', 'has-rash', 'has-sore-throat'],
    'has-measles',
    'Rule: Fever + Rash + Sore-throat → Measles'
)

es.add_rule(
    ['has-fever', 'has-headache', 'has-stiff-neck'],
    'has-meningitis',
    'Rule: Fever + Headache + Stiff-neck → Meningitis'
)

es.add_rule(
    ['has-cough', 'has-wheezing', 'difficulty-breathing'],
    'has-asthma',
    'Rule: Cough + Wheezing + Breathing → Asthma'
)

# Try to diagnose
print("=== MEDICAL DIAGNOSIS EXPERT SYSTEM ===\n")
print("Goal: Determine if patient has flu\n")

# Prove the goal
if es.prove('has-flu'):
    print("\n=== DIAGNOSIS ===")
    print("CONCLUSION: Patient has FLU")
    print("\nREASONING PATH:")
    print("1. Patient has fever (confirmed)")
    print("2. Patient has cough (confirmed)")
    print("3. Patient has body-ache (confirmed)")
    print("4. All conditions match flu symptoms")

```

```

else:
    print("\n=== DIAGNOSIS ===")
    print("CONCLUSION: Patient does NOT have flu")

```

OUTPUT (Interactive):

=== MEDICAL DIAGNOSIS EXPERT SYSTEM ===

Goal: Determine if patient has flu

Trying to prove: has-flu

Trying rule: Rule: Fever + Cough + Body-ache → Flu

Trying to prove: has-fever

No rules found. Asking user...

Does the animal have fever? (yes/no): yes

✓ Proved: has-fever

Trying to prove: has-cough

No rules found. Asking user...

Does the animal have cough? (yes/no): yes

✓ Proved: has-cough

Trying to prove: has-body-ache

No rules found. Asking user...

Does the animal have body ache? (yes/no): yes

✓ Proved: has-body-ache

✓ Proved: has-flu

=== DIAGNOSIS ===

CONCLUSION: Patient has FLU

REASONING PATH:

1. Patient has fever (confirmed)
2. Patient has cough (confirmed)
3. Patient has body-ache (confirmed)
4. All conditions match flu symptoms

FORWARD vs BACKWARD CHAINING:

Aspect	Forward	Backward
Starting Point	Known facts	Goal/hypothesis
Direction	Facts → Conclusions	Goal → Facts
Strategy	Data-driven	Goal-driven
User Questions	Asks everything	Asks only relevant
Efficiency	May derive unused	Only proves goal
Best For	Monitoring, control	Diagnosis, classification

WHEN TO USE BACKWARD CHAINING:

- ✓ Diagnosis (medical, technical)
- ✓ Classification problems
- ✓ Question-answering systems
- ✓ Troubleshooting
- ✓ When you have goal and need supporting facts

Examples:

- Medical diagnosis: Is disease X present?
- Fault diagnosis: Is component Y faulty?
- Credit approval: Should loan be approved?
- Tax advisor: Which deduction applies?

PROJECT 1: MEDICAL DIAGNOSIS EXPERT SYSTEM

Complete Implementation

COMPREHENSIVE MEDICAL DIAGNOSIS SYSTEM

Domain: Common diseases diagnosis

Approach: Backward chaining with certainty factors

Diseases Covered: Flu, Cold, COVID-19, Allergies, Pneumonia

COMPLETE PYTHON IMPLEMENTATION:

```
class MedicalDiagnosisSystem:
    def __init__(self):
        self.facts = {}
        self.rules = []
        self.certainty_factors = {}

    def add_rule(self, name, conditions, conclusion, cf=1.0):
        """Add diagnostic rule with certainty factor"""
        self.rules.append({
            'name': name,
            'conditions': conditions, # List of (symptom, required_cf)
            'conclusion': conclusion,
            'cf': cf
        })

    def ask_symptom(self, symptom, symptom_text):
        """Ask user about symptom"""
        if symptom in self.facts:
            return self.facts[symptom]

        print(f"\n{symptom_text}")
        print("0 = Not present")
        print("1 = Mild")
        print("2 = Moderate")
        print("3 = Severe")

        while True:
            try:
                severity = int(input("Enter severity (0-3): "))
                if 0 <= severity <= 3:
                    cf = severity / 3.0 # Convert to 0.0-1.0
                    self.facts[symptom] = cf
                    return cf
            except:
                print("Invalid input. Try again.")

    def calculate_certainty(self, conditions):
        """Calculate combined certainty factor"""
        cf_product = 1.0

        for symptom, required_cf in conditions:
            actual_cf = self.facts.get(symptom, 0.0)
            if actual_cf < required_cf:
                return 0.0 # Condition not met
            cf_product *= actual_cf

        return cf_product

    def diagnose(self):
        """Run diagnosis"""
        print("=== MEDICAL DIAGNOSIS EXPERT SYSTEM ===")
        print("Please answer questions about symptoms.\n")

        # Ask about symptoms
        symptoms = [
```

```

        ('fever', 'Fever'),
        ('cough', 'Cough'),
        ('sore_throat', 'Sore throat'),
        ('runny_nose', 'Runny nose'),
        ('body_ache', 'Body ache'),
        ('fatigue', 'Fatigue'),
        ('headache', 'Headache'),
        ('loss_of_taste', 'Loss of taste/smell'),
        ('shortness_breath', 'Shortness of breath'),
        ('chest_pain', 'Chest pain')
    ]

    for symptom, text in symptoms:
        self.ask_symptom(symptom, text)

    # Evaluate all diagnostic rules
    diagnoses = []

    for rule in self.rules:
        cf = self.calculate_certainty(rule['conditions'])
        if cf > 0:
            diagnoses.append({
                'disease': rule['conclusion'],
                'certainty': cf * rule['cf'],
                'rule': rule['name']
            })

    # Sort by certainty
    diagnoses.sort(key=lambda x: x['certainty'], reverse=True)

    # Display results
    print("\n" + "="*60)
    print("DIAGNOSTIC RESULTS")
    print("="*60)

    if not diagnoses:
        print("\nNo clear diagnosis based on symptoms.")
        print("Recommend: Consult healthcare professional.")
    else:
        print("\nPOSSIBLE DIAGNOSES (ordered by likelihood):\n")

        for i, diag in enumerate(diagnoses, 1):
            cf_percent = diag['certainty'] * 100
            confidence = "HIGH" if cf_percent > 70 else "MODERATE" if cf_percent > 40 else "LOW"

            print(f"{i}. {diag['disease']}")
            print(f"    Certainty: {cf_percent:.1f}% ({confidence})")
            print(f"    Based on: {diag['rule']}\n")

        # Primary diagnosis
        primary = diagnoses[0]
        print("="*60)
        print(f"PRIMARY DIAGNOSIS: {primary['disease']}")
        print(f"CONFIDENCE: {primary['certainty']*100:.1f}%")
        print("="*60)

        # Recommendations
        self.give_recommendations(primary['disease'])

    def give_recommendations(self, disease):
        """Provide recommendations based on diagnosis"""
        print("\nRECOMMENDED ACTIONS:\n")

        recommendations = {
            'Influenza (Flu)': [
                "1. Rest and stay hydrated",
                "2. Take over-the-counter pain reliever (acetaminophen/ibuprofen)",
                "3. Monitor fever",
                "4. Consider antiviral medication (consult doctor)",
                "5. Isolate to prevent spread",
                "6. Seek medical attention if symptoms worsen"
            ]
        }

```

```

],
'Common Cold': [
    "1. Rest and drink plenty of fluids",
    "2. Use throat lozenges for sore throat",
    "3. Saline nasal drops for congestion",
    "4. Over-the-counter cold medicine if needed",
    "5. Monitor symptoms",
    "6. See doctor if lasts more than 10 days"
],
'COVID-19': [
    "1. ISOLATE IMMEDIATELY",
    "2. Get tested for COVID-19",
    "3. Notify close contacts",
    "4. Monitor oxygen levels if possible",
    "5. Seek immediate care if difficulty breathing",
    "6. Follow local health department guidelines",
    "7. Rest and stay hydrated"
],
'Pneumonia': [
    "1. SEEK MEDICAL ATTENTION IMMEDIATELY",
    "2. May require antibiotics",
    "3. Chest X-ray recommended",
    "4. Monitor breathing and oxygen",
    "5. Complete full course of antibiotics if prescribed",
    "6. Follow up with doctor"
],
'Seasonal Allergies': [
    "1. Identify and avoid allergens",
    "2. Take antihistamines",
    "3. Use nasal corticosteroid spray",
    "4. Keep windows closed during high pollen",
    "5. Shower after being outdoors",
    "6. See allergist for severe cases"
]
}

recs = recommendations.get(disease, ["Consult healthcare professional"])
for rec in recs:
    print(f"    {rec}")

print("\n" + "="*60)
print("DISCLAIMER: This is not a substitute for professional")
print("medical advice. Always consult a healthcare provider.")
print("="*60)

# Initialize system
system = MedicalDiagnosisSystem()

# Add diagnostic rules
system.add_rule(
    'Flu Rule',
    [('fever', 0.5), ('cough', 0.3), ('body_ache', 0.3), ('fatigue', 0.3)],
    'Influenza (Flu)',
    cf=0.9
)

system.add_rule(
    'Cold Rule',
    [('runny_nose', 0.5), ('sore_throat', 0.3), ('cough', 0.2)],
    'Common Cold',
    cf=0.85
)

system.add_rule(
    'COVID-19 Rule',
    [('fever', 0.5), ('cough', 0.5), ('loss_of_taste', 0.7)],
    'COVID-19',
    cf=0.95
)

system.add_rule(

```

```

        'Pneumonia Rule',
        [('fever', 0.6), ('cough', 0.6), ('shortness_breath', 0.5), ('chest_pain', 0.4)],
        'Pneumonia',
        cf=0.9
    )

system.add_rule(
    'Allergy Rule',
    [('runny_nose', 0.5), ('sore_throat', 0.2)],
    'Seasonal Allergies',
    cf=0.8
)

# Run diagnosis
system.diagnose()

```

SAMPLE OUTPUT:

=== MEDICAL DIAGNOSIS EXPERT SYSTEM ===
Please answer questions about symptoms.

Fever
0 = Not present
1 = Mild
2 = Moderate
3 = Severe
Enter severity (0-3): 2

Cough
Enter severity (0-3): 3

[... more questions ...]

=====

DIAGNOSTIC RESULTS

=====

POSSIBLE DIAGNOSES (ordered by likelihood):

1. Influenza (Flu)
Certainty: 78.5% (HIGH)
Based on: Flu Rule
2. Pneumonia
Certainty: 45.2% (MODERATE)
Based on: Pneumonia Rule
3. COVID-19
Certainty: 38.7% (LOW)
Based on: COVID-19 Rule

=====

PRIMARY DIAGNOSIS: Influenza (Flu)

CONFIDENCE: 78.5%

=====

RECOMMENDED ACTIONS:

1. Rest and stay hydrated
2. Take over-the-counter pain reliever
3. Monitor fever
4. Consider antiviral medication (consult doctor)
5. Isolate to prevent spread
6. Seek medical attention if symptoms worsen

=====

DISCLAIMER: This is not a substitute for professional medical advice. Always consult a healthcare provider.

=====

PROJECT 2: CAR TROUBLESHOOTING EXPERT SYSTEM

AUTOMOTIVE FAULT DIAGNOSIS SYSTEM

Domain: Car starting problems

Approach: Backward chaining

Goal: Identify why car won't start

COMPLETE IMPLEMENTATION:

```
class CarTroubleshootingSystem:
    def __init__(self):
        self.facts = {}
        self.rules = self.create_rules()

    def create_rules(self):
        """Define troubleshooting rules"""
        return [
            {
                'name': 'Dead Battery',
                'conditions': ['no_cranking', 'lights_dim', 'clicking_sound'],
                'conclusion': 'dead_battery',
                'solution': 'Jump start or replace battery'
            },
            {
                'name': 'Bad Starter',
                'conditions': ['no_cranking', 'lights_normal', 'no_clicking'],
                'conclusion': 'bad_starter',
                'solution': 'Replace starter motor'
            },
            {
                'name': 'Out of Fuel',
                'conditions': ['cranks_normal', 'fuel_gauge_empty'],
                'conclusion': 'no_fuel',
                'solution': 'Add fuel to tank'
            },
            {
                'name': 'Flooded Engine',
                'conditions': ['cranks_normal', 'smell_gas', 'no_start'],
                'conclusion': 'flooded_engine',
                'solution': 'Wait 10 minutes, try starting with gas pedal to floor'
            },
            {
                'name': 'Bad Spark Plugs',
                'conditions': ['cranks_normal', 'rough_idle', 'no_start'],
                'conclusion': 'bad_spark_plugs',
                'solution': 'Replace spark plugs'
            },
            {
                'name': 'Alternator Failure',
                'conditions': ['starts_then_dies', 'battery_light_on'],
                'conclusion': 'bad_alternator',
                'solution': 'Replace alternator'
            }
        ]

    def ask_yes_no(self, question):
        """Ask yes/no question"""
        while True:
            answer = input(f"{question} (yes/no): ").strip().lower()
```

```

        if answer in ['yes', 'y']:
            return True
        elif answer in ['no', 'n']:
            return False
        print("Please answer yes or no")

def check_symptom(self, symptom):
    """Check if symptom is present"""
    if symptom in self.facts:
        return self.facts[symptom]

    # Map symptom to question
    questions = {
        'no_cranking': "Does the engine fail to crank at all?",
        'lights_dim': "Are the dashboard lights dim or off?",
        'clicking_sound': "Do you hear a clicking sound when turning key?",
        'lights_normal': "Are the dashboard lights normal/bright?",
        'no_clicking': "Is there NO clicking sound?",
        'cranks_normal': "Does the engine crank normally?",
        'fuel_gauge_empty': "Is the fuel gauge showing empty?",
        'smell_gas': "Do you smell gas?",
        'no_start': "Does the engine fail to start?",
        'rough_idle': "Did the car have rough idle before?",
        'starts_then_dies': "Does the car start but then die?",
        'battery_light_on': "Is the battery warning light on?"
    }

    question = questions.get(symptom, f"Symptom: {symptom}?")
    result = self.ask_yes_no(question)
    self.facts[symptom] = result

    return result

def diagnose(self):
    """Run diagnostic process"""
    print("="*70)
    print("CAR TROUBLESHOOTING EXPERT SYSTEM")
    print("="*70)
    print("\nI will help you diagnose why your car won't start.")
    print("Please answer the following questions.\n")

    # Try each rule
    for rule in self.rules:
        # Check all conditions
        all_conditions_met = True

        for condition in rule['conditions']:
            if not self.check_symptom(condition):
                all_conditions_met = False
                break

        if all_conditions_met:
            # Found diagnosis
            print("\n" + "="*70)
            print("DIAGNOSIS FOUND")
            print("="*70)
            print(f"\nPROBLEM: {rule['conclusion'].replace('_', ' ').title()}")
            print(f"\nSOLUTION: {rule['solution']}")
            print(f"\nREASONING: {rule['name']} rule matched")
            print("\nCONDITIONS MET:")
            for cond in rule['conditions']:
                print(f"  ✓ {cond.replace('_', ' ').title()}")

```

```

        print("\n" + "="*70)
        return

    # No diagnosis found
    print("\n" + "="*70)
    print("UNABLE TO DIAGNOSE")
    print("="*70)
    print("\nBased on the symptoms described, I cannot determine")
    print("the exact problem.")
    print("\nRECOMMENDATION: Take car to professional mechanic")
    print("="*70)

# Run system
system = CarTroubleshootingSystem()
system.diagnose()

```

SAMPLE RUN:

```

=====
CAR TROUBLESHOOTING EXPERT SYSTEM
=====

```

I will help you diagnose why your car won't start.
Please answer the following questions.

Does the engine fail to crank at all? (yes/no): yes
Are the dashboard lights dim or off? (yes/no): yes
Do you hear a clicking sound when turning key? (yes/no): yes

```

=====
DIAGNOSIS FOUND
=====

```

PROBLEM: Dead Battery

SOLUTION: Jump start or replace battery

REASONING: Dead Battery rule matched

CONDITIONS MET:

- ✓ No Cranking
- ✓ Lights Dim
- ✓ Clicking Sound

```

=====

```

DECISION TREE VISUALIZATION:

```

Car Won't Start
|
|--- Engine doesn't crank?
|   |
|   |--- Lights dim? → Dead Battery
|   |--- Lights normal? → Bad Starter
|   |
|--- Engine cranks normally?

```

- └ Fuel gauge empty? → Out of Fuel
- └ Smell gas? → Flooded Engine
- └ Rough idle history? → Bad Spark Plugs

PROJECT 3: INVESTMENT ADVISOR EXPERT SYSTEM

FINANCIAL INVESTMENT ADVISORY SYSTEM

Domain: Personal investment recommendations

Features: Risk assessment, portfolio allocation, specific recommendations

IMPLEMENTATION:

```
class InvestmentAdvisor:
    def __init__(self):
        self.user_profile = {}

    def gather_information(self):
        """Collect user information"""
        print("=== INVESTMENT PROFILE QUESTIONNAIRE ===\n")

        # Age
        self.user_profile['age'] = int(input("Your age: "))

        # Investment amount
        amount_str = input("Investment amount ($): ")
        self.user_profile['amount'] = float(amount_str.replace(',', ''))

        # Time horizon
        print("\nInvestment time horizon:")
        print("1. Short-term (< 3 years)")
        print("2. Medium-term (3-10 years)")
        print("3. Long-term (> 10 years)")
        horizon = int(input("Select (1-3): "))
        self.user_profile['horizon'] = ['short', 'medium', 'long'][horizon-1]

        # Risk tolerance
        print("\nRisk tolerance:")
        print("1. Conservative (preserve capital)")
        print("2. Moderate (balanced growth)")
        print("3. Aggressive (maximum growth)")
        risk = int(input("Select (1-3): "))
        self.user_profile['risk'] = ['conservative', 'moderate', 'aggressive'][risk-1]

        # Financial goals
        print("\nPrimary financial goal:")
        print("1. Retirement")
        print("2. Child's education")
        print("3. Wealth accumulation")
        print("4. Income generation")
        goal = int(input("Select (1-4): "))
        goals = ['retirement', 'education', 'wealth', 'income']
        self.user_profile['goal'] = goals[goal-1]

        # Monthly savings capacity
        self.user_profile['monthly_savings'] = float(input("\nMonthly savings capacity ($): "))

    def assess_risk_profile(self):
        """Determine investor risk profile"""
        age = self.user_profile['age']
        horizon = self.user_profile['horizon']
        risk = self.user_profile['risk']

        # Rules for risk assessment
        if age > 60 or horizon == 'short':
            return 'conservative'
        elif age > 45 or risk == 'conservative':
            return 'moderate-conservative'
        elif age < 35 and horizon == 'long' and risk == 'aggressive':
            return 'aggressive'
        elif risk == 'moderate':
            return 'moderate'
        else:
```

```

        return 'moderate-aggressive'

def generate_portfolio(self, risk_profile):
    """Generate portfolio allocation"""
    portfolios = {
        'conservative': {
            'bonds': 60,
            'stocks': 20,
            'cash': 15,
            'real_estate': 5
        },
        'moderate-conservative': {
            'bonds': 45,
            'stocks': 35,
            'cash': 10,
            'real_estate': 10
        },
        'moderate': {
            'bonds': 30,
            'stocks': 50,
            'cash': 5,
            'real_estate': 15
        },
        'moderate-aggressive': {
            'bonds': 20,
            'stocks': 60,
            'cash': 5,
            'real_estate': 15
        },
        'aggressive': {
            'bonds': 10,
            'stocks': 75,
            'cash': 0,
            'real_estate': 15
        }
    }

    return portfolios[risk_profile]

def recommend_specific_investments(self, risk_profile, goal):
    """Recommend specific investment products"""
    recommendations = {
        'conservative': {
            'bonds': ['Government bonds', 'Investment-grade corporate bonds'],
            'stocks': ['Blue-chip dividend stocks', 'Index funds'],
            'cash': ['High-yield savings account', 'Money market funds'],
            'real_estate': ['REITs']
        },
        'aggressive': {
            'bonds': ['High-yield bonds'],
            'stocks': ['Growth stocks', 'Technology sector ETFs', 'Emerging markets'],
            'real_estate': ['Real estate crowdfunding', 'REITs']
        }
    }

    # Simplified - return based on risk profile
    if risk_profile in ['conservative', 'moderate-conservative']:
        return recommendations['conservative']
    else:
        return recommendations['aggressive']

def calculate_projections(self):
    """Project future value"""
    amount = self.user_profile['amount']
    monthly = self.user_profile['monthly_savings']

    # Expected returns by risk profile
    returns = {
        'conservative': 0.04,
        'moderate-conservative': 0.06,
        'moderate': 0.07,

```

```

        'moderate-aggressive': 0.09,
        'aggressive': 0.11
    }

    risk_profile = self.assess_risk_profile()
    annual_return = returns[risk_profile]

    # Calculate for 5, 10, 20 years
    projections = {}
    for years in [5, 10, 20]:
        # Future value of lump sum
        fv_lump = amount * ((1 + annual_return) ** years)

        # Future value of monthly contributions
        months = years * 12
        monthly_rate = annual_return / 12
        fv_monthly = monthly * ((1 + monthly_rate) ** months - 1) / monthly_rate

        total = fv_lump + fv_monthly
        projections[years] = total

    return projections, annual_return

def generate_advice(self):
    """Generate comprehensive investment advice"""
    self.gather_information()

    print("\n" + "="*70)
    print("INVESTMENT RECOMMENDATION REPORT")
    print("="*70)

    # Risk profile
    risk_profile = self.assess_risk_profile()
    print(f"\nRISK PROFILE: {risk_profile.upper().replace('-', ' ')}")

    # Portfolio allocation
    print("\n RECOMMENDED PORTFOLIO ALLOCATION:\n")
    portfolio = self.generate_portfolio(risk_profile)

    amount = self.user_profile['amount']
    for asset, percentage in portfolio.items():
        dollar_amount = amount * (percentage / 100)
        print(f" {asset.title():15} {percentage:3}%    ${dollar_amount:,.2f}")

    # Specific recommendations
    print("\n SPECIFIC INVESTMENT RECOMMENDATIONS:\n")
    recommendations = self.recommend_specific_investments(
        risk_profile,
        self.user_profile['goal']
    )

    for asset_class, products in recommendations.items():
        print(f" {asset_class.title():15}:")
        for product in products:
            print(f"     • {product}")

    # Projections
    print("\n PROJECTED PORTFOLIO VALUE:\n")
    projections, annual_return = self.calculate_projections()

    print(f" Assumed annual return: {annual_return*100:.1f}%\n")
    for years, value in projections.items():
        print(f" After {years:2} years: ${value:,.2f}")

    # Recommendations based on goal
    print("\n ADDITIONAL RECOMMENDATIONS:\n")

    goal = self.user_profile['goal']
    if goal == 'retirement':
        print("     • Max out 401(k) contributions")
        print("     • Consider Roth IRA")

```

```

        print("    • Review and rebalance annually")
    elif goal == 'education':
        print("    • Open 529 education savings plan")
        print("    • Consider age-based portfolio")
    elif goal == 'wealth':
        print("    • Dollar-cost averaging strategy")
        print("    • Tax-loss harvesting")

    print("\n" + "="*70)
    print("DISCLAIMER: This is general advice. Consult a licensed")
    print("financial advisor before making investment decisions.")
    print("="*70)

# Run advisor
advisor = InvestmentAdvisor()
advisor.generate_advice()

```

SAMPLE OUTPUT:

=== INVESTMENT PROFILE QUESTIONNAIRE ===

Your age: 30
Investment amount (\$): 50000
Investment time horizon:
1. Short-term (< 3 years)
2. Medium-term (3-10 years)
3. Long-term (> 10 years)
Select (1-3): 3

Risk tolerance:
1. Conservative (preserve capital)
2. Moderate (balanced growth)
3. Aggressive (maximum growth)
Select (1-3): 3

Primary financial goal:
1. Retirement
2. Child's education
3. Wealth accumulation
4. Income generation
Select (1-4): 1

Monthly savings capacity (\$): 1000

=====

INVESTMENT RECOMMENDATION REPORT

=====

RISK PROFILE: AGGRESSIVE

RECOMMENDED PORTFOLIO ALLOCATION:

Bonds	10%	\$5,000.00
Stocks	75%	\$37,500.00
Cash	0%	\$0.00
Real Estate	15%	\$7,500.00

SPECIFIC INVESTMENT RECOMMENDATIONS:

Bonds:

- High-yield bonds

Stocks:

- Growth stocks
- Technology sector ETFs
- Emerging markets

Real Estate:

- Real estate crowdfunding
- REITs

PROJECTED PORTFOLIO VALUE:

Assumed annual return: 11.0%

After 5 years: \$161,487.29

After 10 years: \$350,713.68

After 20 years: \$1,101,284.52

ADDITIONAL RECOMMENDATIONS:

- Max out 401(k) contributions
- Consider Roth IRA
- Review and rebalance annually

=====

DISCLAIMER: This is general advice. Consult a licensed
financial advisor before making investment decisions.

=====

EXPERT SYSTEMS BEST PRACTICES

DEVELOPMENT BEST PRACTICES:

1. Knowledge Acquisition

- Work closely with domain experts
- Document all rules and reasoning
- Validate knowledge regularly
- Use multiple knowledge elicitation techniques
- Record why decisions were made

2. Rule Design

- Keep rules simple and atomic
- Use consistent naming conventions
- Avoid conflicting rules
- Document each rule clearly
- Test rules individually

3. System Architecture

- Separate knowledge from inference engine
- Make knowledge base modular
- Use appropriate data structures
- Plan for scalability
- Version control knowledge base

4. User Interface

- Make questions clear and unambiguous
- Provide helpful explanations
- Allow users to review/change answers
- Display reasoning process
- Give confidence levels

5. Testing & Validation

- Test with domain expert
- Create comprehensive test cases
- Verify edge cases
- Benchmark against expert performance
- Regular accuracy assessments

6. Maintenance

- Update knowledge regularly

- Track rule effectiveness
- Remove obsolete rules
- Monitor system performance
- Document all changes

CONCLUSION

Congratulations! You have completed the Expert Systems Complete Practical Guide.

What You Have Mastered:

- ✓ Expert systems fundamentals
- ✓ Knowledge representation techniques
- ✓ Forward and backward chaining
- ✓ Uncertainty handling
- ✓ Python implementations
- ✓ Real-world expert systems development
- ✓ Medical diagnosis systems
- ✓ Technical troubleshooting
- ✓ Financial advisory systems
- ✓ Best practices and methodologies

Career Opportunities:

- AI/ML Engineer: \$100,000-\$160,000
- Knowledge Engineer: \$85,000-\$130,000
- Expert System Developer: \$90,000-\$140,000
- AI Consultant: \$110,000-\$180,000
- Intelligent Systems Architect: \$120,000-\$190,000

Next Steps:

1. Build your own expert systems
2. Learn CLIPS programming language
3. Explore machine learning integration
4. Study fuzzy logic systems
5. Research neural-symbolic AI
6. Contribute to open-source AI projects

Applications in Industry:

- Healthcare: Diagnosis, treatment planning
- Finance: Investment advice, risk assessment

- Manufacturing: Quality control, maintenance
- Customer Service: Intelligent chatbots
- Agriculture: Crop management
- Legal: Contract analysis, case research

Remember: Expert systems bridge the gap between human expertise and automated decision-making. Your skills enable you to capture and preserve valuable knowledge, making it accessible 24/7.

Continue learning, keep building, and help create intelligent systems that make a difference!

Happy Expert System Development! 🤖💡

From Human Expertise to Intelligent Systems!